Python Fundamentals

The execution of Python code by the interpreter, the memory layout of a list, list operations from constant to quadratic time, and the cases in which a set or a dictionary is the correct container instead of a list. Standard library only.

01 Python execution model
02 Function definition: is_prime
03 List memory layout
04 Removing elements
05 Membership: list against set
06 Dictionaries
07 Mutability and hashability
08 Operation costs by container

Scope. Python's containers have very different costs for operations that look identical in the source, and choosing the wrong one raises no error: the program gives correct answers and becomes slow as the data grows.

After reading it you should be able to do the following without looking anything up.

- Say what happens between pressing run and seeing output, and why a type error can appear only when a line is reached.
- Write a function with a parameter and a return value, loop, and test it with `assert`.
- State what a list stores, and why indexing is constant time while searching is not.
- Recognise the quadratic form of a loop that removes from the middle, and write the linear one instead.
- Choose between `list`, `set` and `dict` for a counting or lookup job, and justify it.
- Say why a list cannot be a dictionary key and a tuple can.

01

Python execution model

The steps between pressing run and seeing output, and the step at which each kind of error is reported.

EXECUTION STAGES

1. **Lexical and syntactic analysis.** The text is checked against the grammar. A `SyntaxError` at this stage means that no line has run, including the lines above the mistake.
2. **Translation to bytecode**, a compact intermediate form.
3. **Execution.** The interpreter, a virtual machine, reads bytecode instructions one at a time. Each is implemented by a piece of compiled C code.

Most semantic rules, such as whether an operation supports the types of its operands, are checked only during the third step. That is why a type error inside a branch that never runs is never reported.

An ahead-of-time compiler, as used by C or Rust, analyses the whole program before running any of it. That costs time before the program starts, and in return whole classes of error are caught before it runs and the compiler can optimise the code. Python does neither and starts running immediately. The difference in execution speed is large, which is why numerical work in Python is done through libraries whose inner loops are not written in Python.

DYNAMIC TYPING AND STRONG TYPING

The two terms describe different properties.

```
my_variable = 42
my_variable = "abc"        # fine, a name is not tied to a type
```

Dynamic typing means a variable is a name holding a reference to an object, and it can be pointed at an object of any type at any time. The type belongs to the object, never to the name.

```
42 + "42"        # TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Strong typing means Python does not convert an operand implicitly to make that expression work. Some languages produce `"4242"` or `84`. Python raises `TypeError`, and the conversion has to be written explicitly:

```
str(42) + "42"             # -> '4242'   string concatenation
42 + int("42")             # -> 84       numeric addition
```

NOTEBOOK KERNEL STATE

Running a script starts a fresh interpreter every time and discards it at the end. A notebook does not: the kernel keeps running between cells and holds every variable and definition from every cell that has run, in the order the cells ran rather than the order they appear in.

As a result, a cell containing `counter += 1` gives a different answer each time it runs, and a name whose definition has been deleted from a cell is still defined in the kernel, so the notebook keeps working until it is run in a fresh kernel.

> #### KERNEL RESTART AND FULL RERUN
>
> Restarting the kernel and running every cell from the top reproduces what a fresh session does. A notebook that fails then depends on state that existed only in the old kernel.

02

Function definition: is_prime

Write `is_prime(n)`, returning `True` when `n` is prime. Anything at or below 1 is not prime.

```
is_prime(29)  ->  True
is_prime(1)   ->  False
is_prime(4)   ->  False
```

The solution below contains a function definition with a parameter and a return value, indentation as block structure, conditionals, a loop, an operator, an import, and a way to test the result.

```
def is_prime(n):
    if n <= 1:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False
    i = 3                       # check odd divisors only
    while i * i <= n:
        if n % i == 0:
            return False
        i += 2
    return True
```

Notes on the code above:

- **The bound.** Testing divisors up to `n` is wasteful. If `n` has a divisor larger than its square root it must also have the matching one below it, so testing up to the square root finds a divisor whenever one exists. `i * i <= n` expresses that without computing a square root.
- **Even numbers are handled once**, up front, which is what allows the loop to step by 2.
- **The edge cases.** Zero, one and negative numbers are where a first attempt usually fails, and it fails without an error, by returning a plausible answer.

THE SAME FUNCTION WITH A FOR LOOP

Every Python `for` loop is a for-each loop. There is no C-style three-part form. A counting loop iterates over an object that produces numbers, such as `range(start, stop, step)`, which excludes the stop value.

```
import math

def is_prime(n):
    if n <= 1:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False
    end = int(math.sqrt(n)) + 1     # int() truncates, so + 1 keeps the last candidate
    for i in range(3, end, 2):
        if n % i == 0:
            return False
    return True
```

`math.sqrt` returns a float and `range` requires integers, so the conversion is not optional. Leaving it out raises `TypeError`. The `+ 1` is there because `int()` truncates towards zero: for a perfect square such as 49, `int(math.sqrt(49))` is exactly 7, and without the `+ 1` the exclusive upper bound would skip the largest candidate divisor.

ASSERT

`assert condition, message` does nothing when the condition is true and raises `AssertionError` with the message when it is false. It is the simplest form of test: the program stops at the first wrong result.

```
assert is_prime(2) is True,   "2 is the smallest prime"
assert is_prime(29) is True,  "29 is prime"
assert is_prime(1) is False,  "1 is not prime"
assert is_prime(0) is False,  "0 is not prime"
assert is_prime(-7) is False, "negative numbers are not prime"
assert is_prime(9) is False,  "9 = 3 x 3"
print("all tests passed")
```

INDENTATION

Indentation is how Python delimits a block, the role braces have in C and Java. Indentation has to be consistent within a block. Mixing tabs and spaces can produce code that looks correct and parses as something else. Four spaces is the convention.

03

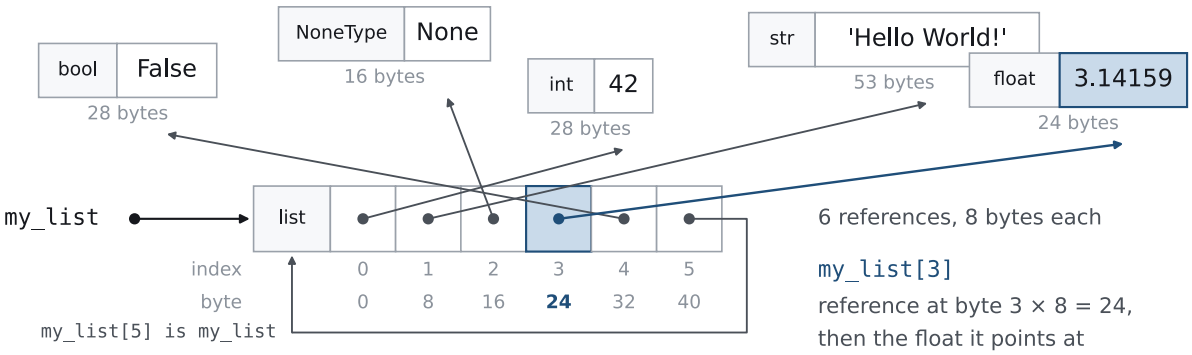
List memory layout

A list stores references, not values. Both the mixed element types a list allows and its memory and speed costs follow from that.

```
my_list = [42, "Hello World!", None, 3.14159, False]   # 5 references
my_list.append(my_list)                                # now 6, the last one to itself
my_list          # -> [42, 'Hello World!', None, 3.14159, False, [...]]
```

A list stores a contiguous block of references, that is, memory addresses of objects that live elsewhere. That is why the element types need not agree, and why a list can contain itself: it holds its own address, not a copy of itself, so there is no infinite regress. The `[...]` is how Python prints a list that contains itself.

The block being contiguous is what makes indexing fast. To find item *i*, Python does arithmetic on the starting address rather than walking the list, so `x[i]` costs the same whether the list holds ten items or ten million.



my_list after the append: 6 references of 8 bytes each, pointing at objects stored elsewhere, the last one at the list itself. Reading my_list[3] takes the reference at byte 24, then the float it points at; object sizes are as sys.getsizeof reports them on Python 3.14.

This has three costs, which grow with the size of the data:

- Reading one element means two memory accesses, one for the reference and one for the object it points at.
- Every element is a full Python object with its own header, so a list of a million integers takes far more memory than the raw integer values. On a 64-bit build of Python 3.14 a small `int` takes 28 bytes, so the references alone are 8 MB and the objects add roughly 28 MB more.
- Those objects are scattered in memory rather than packed together, which makes poor use of the processor cache.

An array library such as NumPy removes this cost, by storing raw values of a single type packed end to end instead of references to scattered objects.

04

Removing elements

Given a list of integers, remove every negative value.

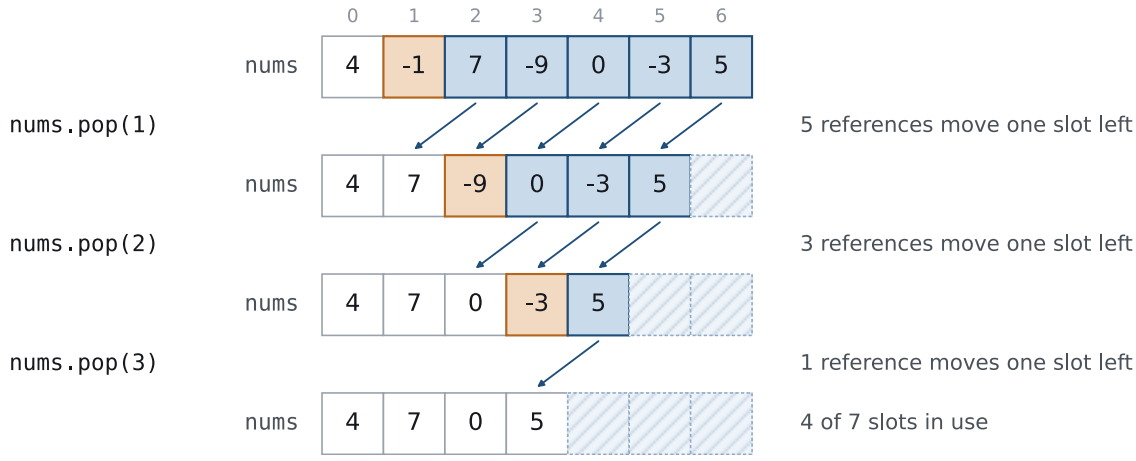
```
[4, -1, 7, -9, 0, -3, 5]  ->  [4, 7, 0, 5]
```

The direct approach walks the list and deletes the negative values. It gives the right answer in quadratic time.

```
def remove_negatives(numbers):
    nums = numbers.copy()          # a copy, so the caller's list is unchanged
    i = 0
    while i < len(nums):
        if nums[i] < 0:
            nums.pop(i)            # do not advance i, the next element slid into this slot
        else:
            i += 1
    return nums
```

A `while` loop is needed rather than a `for` loop, because the length changes during the loop and iterating over a list while modifying it is unreliable.

The quadratic time comes from `pop`. Elements sit in a contiguous block, so removing one from the middle leaves a hole that would break the address arithmetic. Python closes it by shifting every later element down one place, which is `O(n)` work for a single removal. Doing that once per element makes the function `O(n²)`.



The pops made by remove_negatives([4, -1, 7, -9, 0, -3, 5]). Each removes the orange element and moves every later reference, in blue, one slot left: 5, 3 and 1 moves; hatched slots are allocated but unused.

Building a new list instead touches each element once:

```
def remove_negatives(numbers):
    kept = []
    for num in numbers:
        if num >= 0:
            kept.append(num)
    return kept

kept = [num for num in numbers if num >= 0]      # the same thing, as a comprehension
```

`append` adds at the end, where there is no shifting to do. A list keeps spare capacity beyond its current length, so most appends write into a free slot. When it fills, Python allocates a larger block and copies everything across, but since each new block is proportionally bigger those copies grow rarer as the list grows. Averaged over many appends the cost per append is constant, which is what amortised `O(1)` means. The whole function is `O(n)`.

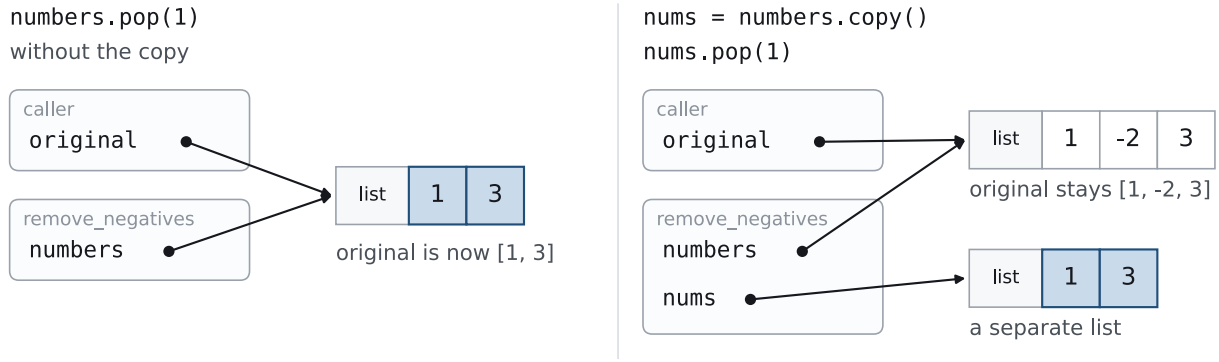
Both versions give the same answers. Removing the negatives from a list of n numbers, best of several runs on Apple silicon with Python 3.14:

N	POP IN A LOOP	BUILD A NEW LIST	RATIO
10 000	0.0034 s	0.00015 s	22×
60 000	0.140 s	0.00088 s	159×
200 000	1.56 s	0.0031 s	512×
500 000	9.69 s	0.0076 s	1275×

The absolute seconds depend on the machine and the Python version. The ratio in the last column grows with the input. When n grows by some factor, the slow version's time grows by roughly the square of that factor and the fast version's by roughly the factor itself. That is what `O(n²)` against `O(n)` means. On a test case of seven elements the difference does not show.

ARGUMENT PASSING

The name `numbers` inside the function refers to the same list object as the name at the call site, so calling `pop` on it changes the caller's list. A function that modifies its argument changes data the caller still uses, which is why the slow version starts with `numbers.copy()`.



remove_negatives(original) with original = [1, -2, 3]. Without the copy, original and numbers name one list and numbers.pop(1) changes it for both; with nums = numbers.copy(), pop changes the copy and original stays [1, -2, 3].

The fast version needs no copy, because it never modifies the input. Building a new value leaves the argument unchanged.

```
original = [1, -2, 3]
result = remove_negatives(original)   # -> [1, 3]
original                              # -> [1, -2, 3], untouched
```

05

Membership: list against set

Given a log of student IDs, find every ID that appears more than once, each reported once.

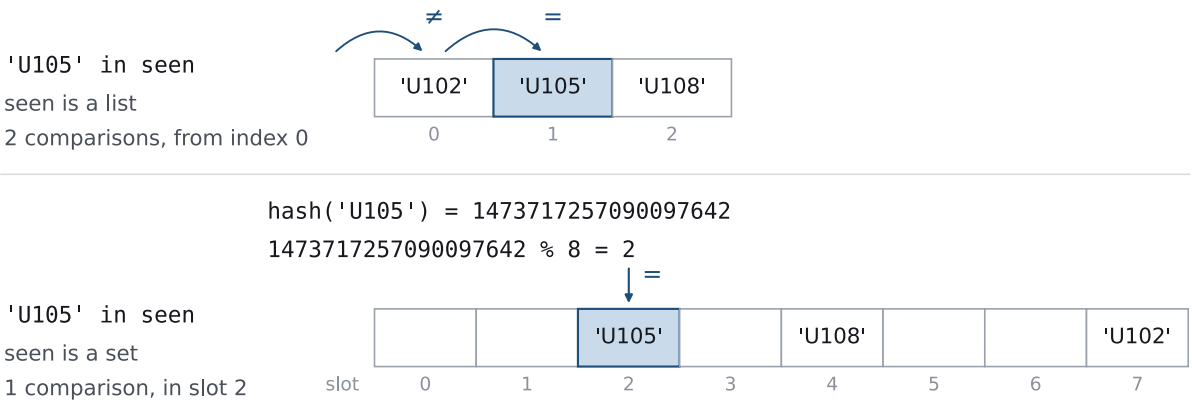
```
submissions = ["U102", "U105", "U102", "U108", "U105", "U101"]
find_duplicates(submissions)   # -> U102 and U105
```

The direct approach keeps a list of the IDs already processed:

```
def find_duplicates(submissions):
    seen = []
    duplicates = []
    for student_id in submissions:
        if student_id in seen:                    # a linear scan
            if student_id not in duplicates:      # another linear scan
                duplicates.append(student_id)
        else:
            seen.append(student_id)
    return duplicates                             # -> ['U102', 'U105']
```

The answers are correct, and the quadratic time comes from `in`. Testing membership of a list compares against elements one at a time, so it is `O(n)` on average, and `seen` grows as the loop proceeds. An `O(n)` lookup once per element makes the function $O(n^2)$.

Lookup can be made faster by sorting or by hashing. Sorting first costs `O(n log n)` and allows a binary search afterwards. Hashing computes a number from the value itself and uses it to jump straight to where the value would be, which is `O(1)` on average. Python's `set` and `dict` both do the second.



The test 'U105' in seen at the fifth submission: the list compares from index 0 until an element matches, the set takes hash('U105') modulo its 8 slots and compares only the key in slot 2. String hashes, and so the slots, differ from one run of Python to the next.

```
def find_duplicates(submissions):
    seen = set()                  # {} would create a dict, not a set
    duplicates = set()
    for student_id in submissions:
        if student_id in seen:    # constant time on average
            duplicates.add(student_id)
        else:
            seen.add(student_id)
    return duplicates             # -> {'U102', 'U105'}, in either order
```

With `set()` in place of `[]` the function is `O(n)`. A membership test against a list inside a loop can be replaced by a test against a set in the same way.

SET

A set behaves like a mathematical one.

- It holds no duplicates. Adding an element that is already present has no effect and raises no error.
- It is unordered. Iteration order does not follow the insertion order and can change after any element is added or removed.

```
xs = [3, 1, 3, 2, 1]
set(xs)                  # -> {1, 2, 3}   the distinct values
len(set(xs)) != len(xs)  # -> True        xs contains duplicates
```

EMPTY CONTAINERS

`{}` is an empty dictionary, not an empty set: set literals were added to Python after dictionary literals, and `{}` stayed a dictionary. An empty set is written `set()`. Non-empty set literals are unambiguous, so `{"a", 42}` is a set.

06

Dictionaries

A set records whether a value is present. A dictionary stores a value under each key.

A dictionary holds key and value pairs. Keys are unique, and looking one up uses the same hash mechanism a set uses, so it is `O(1)` on average.

```
empty = {}                          # or dict()
stock = {"Produce": ["Apples", "Bananas"],
         "Dairy":   ["Milk", "Cheese"],
         "Bakery":  []}             # 3 keys, each value a list
```

ITERATING KEYS, VALUES AND ITEMS

Each entry is a pair, so a loop can run over the keys, the values or the pairs. Looping over the dictionary itself gives the keys.

```
# stock: 3 keys, each value a list

for key in stock:                   # keys, same as stock.keys()
    print(key)

for value in stock.values():        # values only
    print(value)

for key, value in stock.items():    # both at once
    print(key, value)
```

ADDING, UPDATING AND DELETING ENTRIES

Assignment by key inserts an entry when the key is new and overwrites the value when the key exists. A mistyped key therefore creates a second entry instead of updating the intended one.

```
# stock: 3 keys, each value a list

stock["Frozen"] = ["Peas"]          # inserts, the key was new, now 4 keys
stock["Frozen"] = ["Chips"]         # overwrites, the key existed, still 4 keys
del stock["Frozen"]                 # removes the pair, back to 3 keys
"Frozen" in stock                   # -> False, membership tests the key, not the value
```

Since Python 3.7 a dictionary preserves the order keys were first inserted, and iteration follows that order. A set does not preserve insertion order. Values may be of any type. Keys may not: a list cannot be a key.

07

Mutability and hashability

A list cannot be a dictionary key and a tuple can. The difference comes from mutability.

A dictionary can record how long a drone took to fly a route, keyed by the route itself, a sequence of waypoints. A list of coordinates as the key raises an error:

```
costs = {}
route = [[15.5, 27.2], [19.5, 21.4], [22.3, 18.1]]   # a list of 3 lists
costs[route] = 29                   # TypeError: cannot use 'list' as a dict key
                                    # (unhashable type: 'list')
```

MUTABLE AND IMMUTABLE

An object is **mutable** if its state can change after it is created, and **immutable** if it cannot.

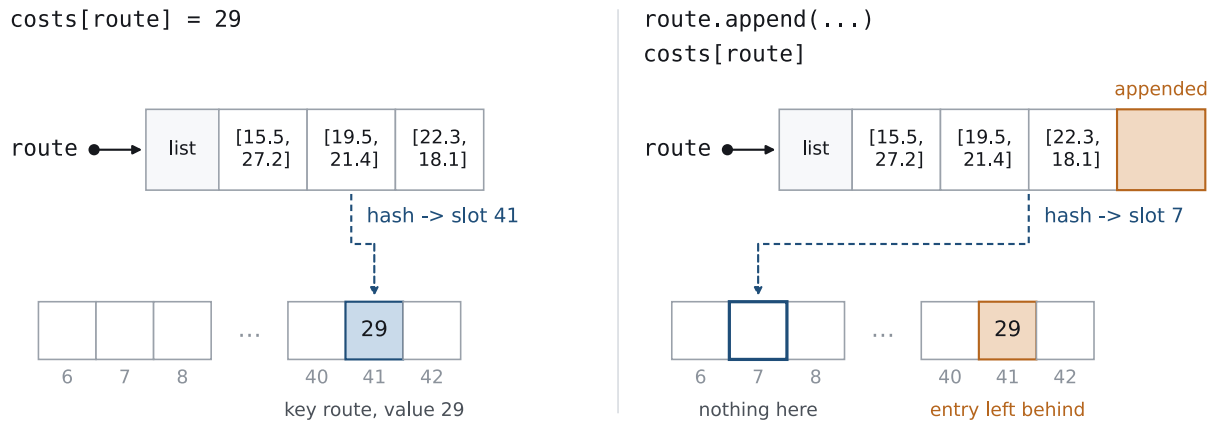
```
values = [1, 2]
values.append(3)        # fine, a list is mutable, so it can be changed in place

text = "abcd"
text[1] = "X"           # TypeError: 'str' object does not support item assignment
```

Strings are immutable. Anything that looks like changing one, such as concatenation or replacing a character, builds a new string and leaves the original unchanged.

HASH LOOKUP WITH MUTABLE KEYS

Looking up a key means hashing it to find the slot to examine. Suppose a list were allowed as a key. It hashes to slot 41 and is stored there. Code elsewhere that still holds a reference to the same list appends to it. Its hash is now different, say slot 7. The entry has become unreachable: a lookup goes to slot 7 and finds nothing, while the entry stays in slot 41.



If lists could be keys, with the slot numbers from the text: costs[route] = 29 would store the entry in slot 41, and after route.append(...) a lookup would go to slot 7 and find nothing. Python raises TypeError for a list key, which prevents this.

Python prevents this. The mutable built-in types have no usable hash, so they cannot be dictionary keys or set members.

TYPE	MUTABLE	HASHABLE	KEY OR SET MEMBER
`int`, `float`, `str`, `bool`	no	yes	yes
`tuple`	no	if its contents are	yes
`frozenset`	no	yes	yes
`list`	yes	no	no
`dict`, `set`	yes	no	no

TUPLES

A tuple is Python's immutable sequence, written with parentheses. A tuple of tuples can be used as the route key.

```
costs = {}
route = ((15.5, 27.2), (19.5, 21.4), (22.3, 18.1))   # a tuple of 3 tuples
costs[route] = 29                 # works

single = (42,)                    # the comma makes it a tuple, (42) is the int 42
route[0] = (0, 0)                 # TypeError: 'tuple' object does not
                                  # support item assignment
```

As the table shows, a tuple is hashable only if everything inside it is, because its hash is computed from its contents. `(1, 2)` can be a key. `(1, [2])` raises the same `TypeError`, because the list inside it can still change.

PACKING AND UNPACKING

Python packs comma-separated values into tuples and unpacks tuples into separate names.

```
a, b = 1, 2                       # both sides are tuples
a, b = b, a                       # a swap with no temporary variable

# stock: 3 keys, each value a list
for key, value in stock.items():  # items() yields 2-tuples, unpacked into two names
    ...

def minmax(xs):
    return min(xs), max(xs)       # one tuple is returned, not two values

low, high = minmax([3, 1, 4])     # -> 1 and 4
```

Unpacking requires the counts to match on both sides. When they do not, Python raises `ValueError`.

08

Operation costs by container

The cost of each common operation on each built-in container, with the qualifiers that apply to it.

OPERATION	LIST	TUPLE	SET	DICT
Read by index, `x[3]`	`O(1)`	`O(1)`	no	no
Read by key, `x[k]`	no	no	no	`O(1)` avg
Membership, `v in x`	`O(n)`	`O(n)`	`O(1)` avg	`O(1)` avg
Add at the end	`O(1)` amortised	no	`O(1)` avg	`O(1)` avg
Insert or delete at the front	`O(n)`	no	not ordered	`O(1)` avg
Keeps insertion order	yes	yes	no	yes
Allows duplicates	yes	yes	no	keys no, values yes
Can be a key or set member	no	if contents are	no	no

A membership test inside a loop over n elements therefore takes quadratic time in total with a list and linear time with a set.

AVERAGE AND AMORTISED

Average: if many keys land in the same slot they collide, and lookup degrades towards scanning that slot. With ordinary data and Python's own hash functions this rarely happens, which is why the average is the figure quoted.

Amortised: most appends write into spare capacity, an occasional one triggers a copy of the whole list, and because the new block is proportionally larger those copies get rarer as the list grows. Averaged over many appends, the cost per append is constant.

FURTHER READING

- The tutorial, for reading through a topic.
- The library reference and the language reference, for the exact rules.
- The built-in functions, documented on a single page, including `len`, `range`, `sum`, `min`, `max`, `sorted`, `enumerate`, `zip`.